



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

EE1003

Design and Implementation of a Digital Clock using Arduino Uno, IC-7447 and Seven-Segment Displays

Author

MANIKANTA D-EE24BTECH11013

Contents

1	Introduction	2
2	Components Used	2
2.1	Arduino Uno	2
2.2	Breadboard	2
2.3	Jumper Cables	2
2.4	Six Seven-Segment Displays	2
2.5	7447 IC	2
2.6	Resistors	2
3	Role of Components in Daily Life	2
3.1	Arduino Uno	2
3.2	Breadboard	2
3.3	Jumper Cables	3
3.4	Seven-Segment Displays	3
3.5	7447 IC	3
3.6	Resistors	3
4	Procedure	3
4.1	Step 1: Setting Up the Breadboard	3
4.2	Step 2: Connecting the 7447 IC	3
4.3	Step 3: Adding Resistors	3
4.4	Step 4: Programming the Arduino Uno	3
4.5	Step 5: Testing the Clock	3
5	Code Implementation and Explanation	4
5.1	Code Listing	4
5.2	Code Explanation	6
6	Conclusion	6
7	Images	6

Abstract

This report details the design and implementation of a digital clock using an Arduino Uno, six seven-segment displays, a 7447 IC, 270-ohm resistors, and a breadboard. The project demonstrates the integration of hardware components to create a functional digital clock. The report explains the role of each component, the procedure for assembling the clock, and the significance of such projects in daily life.

1 Introduction

Digital clocks are ubiquitous in modern life, found in devices ranging from microwaves to smartphones. This project aims to build a simple digital clock using basic electronic components. The clock displays hours and minutes using six seven-segment displays controlled by an Arduino Uno.

2 Components Used

2.1 Arduino Uno

The Arduino Uno is a microcontroller board based on the ATmega328P. It serves as the brain of the clock, controlling the seven-segment displays and managing timekeeping.

2.2 Breadboard

A breadboard is used to create temporary electrical connections between components. It allows for easy prototyping without soldering.

2.3 Jumper Cables

Jumper cables are used to connect components on the breadboard to the Arduino Uno. They provide a flexible and reusable way to make electrical connections.

2.4 Six Seven-Segment Displays

Seven-segment displays are used to display numeric digits. Each display consists of seven LEDs arranged in a specific pattern to represent numbers from 0 to 9. Six displays are used to show hours (two digits), minutes (two digits), and seconds (two digits).

2.5 7447 IC

The 7447 is a BCD-to-seven-segment decoder/driver IC. It converts binary-coded decimal (BCD) input from the Arduino into signals that drive the seven-segment displays.

2.6 Resistors

Six 270-ohm resistors are used to limit the current flowing through the LEDs in the seven-segment displays, preventing them from burning out.

3 Role of Components in Daily Life

3.1 Arduino Uno

Arduino boards are widely used in prototyping and DIY projects. They are used in home automation, robotics, and educational projects.

3.2 Breadboard

Breadboards are essential for testing and prototyping electronic circuits. They are used in educational settings and by hobbyists.

3.3 Jumper Cables

Jumper cables are used in various electronic projects to connect components. They are reusable and make circuit assembly easier.

3.4 Seven-Segment Displays

Seven-segment displays are used in digital clocks, calculators, and other devices that display numeric information.

3.5 7447 IC

The 7447 IC is used in devices that require numeric displays, such as digital clocks and counters.

3.6 Resistors

Resistors are fundamental components in electronics, used to control current and voltage levels in circuits. The 270-ohm resistors ensure the LEDs in the seven-segment displays operate within safe current limits.

4 Procedure

4.1 Step 1: Setting Up the Breadboard

1. Place the six seven-segment displays on the breadboard.
2. Connect the common cathode (or anode) of each display to the ground (or VCC) using jumper cables.

4.2 Step 2: Connecting the 7447 IC

1. Place the 7447 IC on the breadboard.
2. Connect the BCD input pins (A, B, C, D) of the 7447 IC to the digital output pins of the Arduino Uno (e.g., pins 2, 3, 4, 5).
3. Connect the output pins (a, b, c, d, e, f, g) of the 7447 IC to the corresponding segments of the seven-segment displays.

4.3 Step 3: Adding Resistors

1. Connect a 270-ohm resistor between each segment of the seven-segment displays and the output pins of the 7447 IC.
2. This limits the current through the LEDs, protecting them from damage.

4.4 Step 4: Programming the Arduino Uno

1. Write a program in the Arduino IDE to control the seven-segment displays.
2. The program should increment the time every second and update the displays accordingly.
3. Use the 'millis()' function for accurate timekeeping without blocking the code.
4. Upload the program to the Arduino Uno.

4.5 Step 5: Testing the Clock

1. Power the Arduino Uno using a USB cable or an external power supply.
2. Verify that the seven-segment displays show the correct time (HH:MM:SS).
3. Adjust the program if necessary to correct any errors.

5 Code Implementation and Explanation

The following C code is used to control the digital clock. It is written for the Arduino Uno and uses the AVR library for low-level hardware control.

5.1 Code Listing

Listing 1: Arduino Code for Digital Clock

```
1  #include <avr/io.h>
2  #include <avr/interrupt.h>
3  #include <util/delay.h>
4
5  // BCD Output Pins
6  #define A PD2
7  #define B PD3
8  #define C PD4
9  #define D PD5
10
11 // Common Display Pins
12 #define H1 PD6
13 #define H2 PD7
14 #define M1 PB0
15 #define M2 PB1
16 #define S1 PB2
17 #define S2 PB3
18
19 // Global BCD digits for the clock
20 volatile uint8_t h1 = 0, h2 = 0, m1 = 0, m2 = 0, s1 = 0, s2 = 0;
21
22 // Multiplexing control variables
23 uint8_t current_digit = 0;
24 volatile uint32_t millis_count = 0;
25 const uint8_t mux_interval = 2; // 2ms per digit refresh
26
27 // Timekeeping control
28 volatile uint32_t last_second = 0;
29
30 void init_timer0() {
31     TCCR0A |= (1 << WGM01);
32     TCCR0B |= (1 << CS01) | (1 << CS00);
33     OCR0A = 249;
34     TIMSK0 |= (1 << OCIE0A);
35     sei();
36 }
37
38 ISR(TIMERO_COMPA_vect) {
39     millis_count++;
40 }
41
42 uint32_t millis() {
43     uint32_t ms;
44     cli();
45     ms = millis_count;
46     sei();
47     return ms;
48 }
49
50 uint8_t bcdIncrement(uint8_t bcd, uint8_t max) {
51     if (bcd == max) return 0;
52     uint8_t d0 = bcd & 0x01, d1 = (bcd >> 1) & 0x01, d2 = (bcd >> 2) & 0x01, d3 = (bcd >> 3) & 0x01;
53     uint8_t n0 = !d0, c0 = d0, n1 = d1 ^ c0, c1 = d1 & c0, n2 = d2 ^ c1, c2 = d2 & c1, n3 = d3 ^ c2, c3 = d3 & c2;
54     return (n3 << 3) | (n2 << 2) | (n1 << 1) | n0;
```

```

55 }
56
57 void updateTime() {
58     if (millis() - last_second >= 1000) {
59         last_second += 1000;
60         s2 = bcdIncrement(s2, 9);
61         if (s2 == 0) {
62             s1 = bcdIncrement(s1, 5);
63             if (s1 == 0) {
64                 m2 = bcdIncrement(m2, 9);
65                 if (m2 == 0) {
66                     m1 = bcdIncrement(m1, 5);
67                     if (m1 == 0) {
68                         h2 = bcdIncrement(h2, 9);
69                         if (h2 == 0) {
70                             h1 = bcdIncrement(h1, 2);
71                             if (h1 == 2 && h2 > 3) {
72                                 h1 = h2 = 0;
73                             }
74                         }
75                     }
76                 }
77             }
78         }
79     }
80 }
81
82 void displayDigit(uint8_t digit, uint8_t position) {
83     PORTD &= ~((1 << H1) | (1 << H2));
84     PORTB &= ~((1 << M1) | (1 << M2) | (1 << S1) | (1 << S2));
85
86     PORTD &= ~((1 << A) | (1 << B) | (1 << C) | (1 << D));
87     PORTD |= ((digit & 0x01) << A) |
88             ((digit & 0x02) << (B-1)) |
89             ((digit & 0x04) << (C-2)) |
90             ((digit & 0x08) << (D-3));
91
92     switch (position) {
93         case 0: PORTD |= (1 << H1); break;
94         case 1: PORTD |= (1 << H2); break;
95         case 2: PORTB |= (1 << M1); break;
96         case 3: PORTB |= (1 << M2); break;
97         case 4: PORTB |= (1 << S1); break;
98         case 5: PORTB |= (1 << S2); break;
99     }
100
101     _delay_ms(mux_interval);
102 }
103
104 void multiplexDisplay() {
105     uint8_t digits[6] = {h1, h2, m1, m2, s1, s2};
106     displayDigit(digits[current_digit], current_digit);
107     current_digit = (current_digit + 1) % 6;
108 }
109
110 void setup() {
111     DDRD |= (1 << A) | (1 << B) | (1 << C) | (1 << D) | (1 << H1) | (1 << H2);
112     DDRB |= (1 << M1) | (1 << M2) | (1 << S1) | (1 << S2);
113
114     init_timer0();
115
116     last_second = millis();
117 }

```

```

118
119 int main(void) {
120     setup();
121
122     while (1) {
123         updateTime();
124         multiplexDisplay();
125     }
126
127     return 0;
128 }

```

5.2 Code Explanation

1. Initialization:

- The 'setup()' function configures the Arduino pins as outputs and initializes Timer0 for time-keeping.
- The 'init_{timer0}()' *function sets up Timer0 in CTC mode to generate an interrupt every 1ms.*

2. Timekeeping:

- The 'updateTime()' function increments the seconds, minutes, and hours every second using the 'millis()' function.
- The 'bcdIncrement()' function increments a BCD digit and resets it to 0 when it exceeds the maximum value (e.g., 9 for seconds and minutes, 2 for hours).

3. Display Multiplexing:

- The 'multiplexDisplay()' function cycles through the six seven-segment displays, updating each one every 2ms.
- The 'displayDigit()' function sends the BCD value to the 7447 IC and activates the corresponding display.

4. Main Loop:

- The 'main()' function continuously updates the time and refreshes the displays.

6 Conclusion

This project successfully demonstrates the construction of a digital clock using an Arduino Uno, six seven-segment displays, and other electronic components. The project highlights the importance of understanding basic electronics and programming in creating functional devices. The use of 270-ohm resistors ensures the longevity of the LEDs, while the 7447 IC simplifies the control of the seven-segment displays.

7 Images

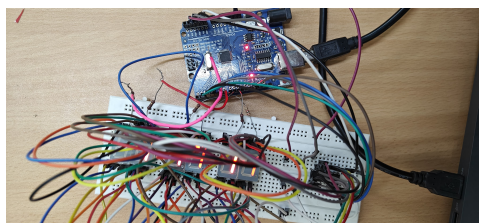


Figure 1: Top view of the assembled clock.

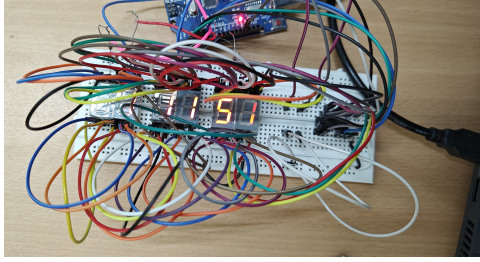


Figure 2: View showing the connections.

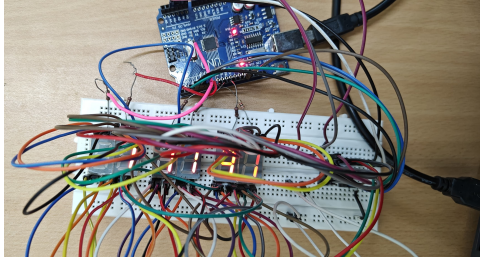


Figure 3: Close-up of the seven-segment displays.

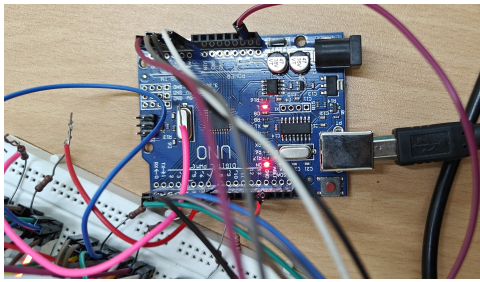


Figure 4: Arduino Uno setup.